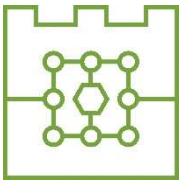


Politechnika Krakowska
im. Tadeusza Kościuszki

Wydział Informatyki i Matematyki



Wiktor Zmiendak

Numer albumu: 142706

Sprawozdanie

Projekt 2

Blum-Blum-Shub generator

Cel zadanie

Celem projektu było pokazanie idei szyfru doskonałego. W klasycznym rozumieniu powinien on być losowy oraz do jednorazowego użytku. Na potrzeby tego laboratorium posłużono się pseudolosowym generatorem BBS.

Implementacja szyfru

W pierwszej kolejności czytamy naszą wiadomość, którą chcemy zaszyfrować i zamieniamy ją na ciąg bitów.

```python
def read_message(file_path: Path) -> str:
    return file_path.read_text(encoding="utf-8")


def text_to_bits(text: str) -> List[int]:
    data = text.encode("utf-8")
    return [(byte >> shift) & 1 for byte in data for shift in range(7, -1, -1)]
```

Następnie uruchamiany jest generator Blum-Blum-Shub, który losuje dwie duże liczby pierwsze spełniające warunek p = 3mod4 oraz q = 3mod4, oblicza moduł n = p * q, wybiera ziarno względnie pierwsze z n oraz iteracyjnie wyznacza kolejne stany $x_i = x_{i-1}$^2modN. Każdy bit klucza powstaje jako najmłodszy bit stanu, a generowanie trwa do momentu uzyskania długości równej długości wiadomości bitowej.

```python
def blum_blum_shub(
    bit_count: int, prime_bits: int = PRIME_BITS
) -> Tuple[List[int], int, int, int, int]:
    p = generate_blum_prime(prime_bits)
    q = generate_blum_prime(prime_bits)
    while q == p:
        q = generate_blum_prime(prime_bits)

    n = p * q
    seed = generate_coprime_seed(n)

    x = pow(seed, 2, n)
    key_bits: List[int] = []
    for _ in range(bit_count):
        x = pow(x, 2, n)
        key_bits.append(x % 2)

    return key_bits, p, q, n, seed
```

Po wygenerowaniu klucza wykonywane są dwa testy z rodziny NIST:

- Test Monobit - sprawdza, czy liczba jedynek i zer jest odpowiednio zbalansowana,
- Test Runs - ocenia, czy liczba przejść między zerami i jedynkami mieści się w akceptowalnym zakresie.

Dla obu testów zastosowano próg decyzyjny, co pozwala odrzucić ciągi o wyraźnie nielosowym charakterze.

```python
def monobit_test(bits: List[int]) -> Tuple[bool, float]:
    n = len(bits)
    if n == 0:
        return False, 0.0

    s_n = sum(1 if bit == 1 else -1 for bit in bits)
    s_obs = abs(s_n) / math.sqrt(n)
    p_value = math.erfc(s_obs / math.sqrt(2))
    return p_value >= P_VALUE_THRESHOLD, p_value
```

```python
def runs_test(bits: List[int]) -> Tuple[bool, float, str]:
    n = len(bits)
    if n == 0:
        return False, 0.0, "Pusty ciag bitow"

    pi = sum(bits) / n
    tau = 2 / math.sqrt(n)
    if abs(pi - 0.5) >= tau:
        return False, 0.0, "Warunek wstepny runs nie jest spelniony"

    runs_count = 1 + sum(1 for i in range(1, n) if bits[i] != bits[i - 1])
    numerator = abs(runs_count - 2 * n * pi * (1 - pi))
    denominator = 2 * math.sqrt(2 * n) * pi * (1 - pi)
    p_value = math.erfc(numerator / denominator)
    return p_value >= P_VALUE_THRESHOLD, p_value, "OK"
```

W celu odszyfrowania wiadomości wykorzystujemy tę samą operację XOR.

Podsumowanie

Implementacja pokazuje, że operacja XOR w połączeniu z kluczem o odpowiedniej długości umożliwia poprawne szyfrowanie i odszyfrowanie wiadomości, natomiast testy dostarczają podstawowej oceny jakości klucza. Jednocześnie należy podkreślić, że dwa testy NIST nie dają pełnego dowodu losowości, lecz stanowią użyteczną kontrolę wstępną.